DATA STRUCTURES AND ALGORITHMS

UNIT III – LINKED LIST

Food Order Management System Using Singly Linked List

Project Report

1. Title

Food Order Management System Using Singly Linked List

2. Problem Statement

Develop a Food Order Management System using a Singly Linked List in C++. Each node represents a food order and stores the Order ID, Food Item, Quantity, and a pointer to the next order.

The system allows the user to add a new food order, delete a completed/cancelled order, search for an order using Order ID, and display all pending orders. The project demonstrates dynamic memory allocation, pointers, linked-list insertion, deletion, searching, and traversal.

3. Objective

• To understand the concept of a Singly Linked List.

• To implement dynamic memory allocation using pointers.

• To perform insertion and deletion operations in a linked list.

• To search for a particular food order using its Order ID.

• To display all pending food orders using linked-list traversal.

• To understand the practical application of linked lists in a real-world system.

4. Theory – Singly Linked List

A Singly Linked List is a linear data structure made up of nodes. Each node contains data and a pointer to the next node. Unlike an array, linked-list nodes do not need contiguous memory locations. Nodes can be created dynamically as required.

In this project, each node stores three data fields—Order ID, Food Item, and Quantity—and one pointer field called next. The head pointer stores the address of the first node.

Basic operations demonstrated: insertion at the end, deletion by Order ID, searching by Order ID, and traversal for displaying all orders.

5. Algorithm

Step 1: Start.

Step 2: Create a Node structure containing Order ID, Food Item, Quantity, and a pointer to the next node.

Step 3: Initialize head = NULL.

Step 4: Display the menu: Add Order, Delete Order, Search Order, Display Orders, Exit.

Step 5: For Add Order: create a new node dynamically, enter order details, and insert it at the end of the linked list.

Step 6: For Delete Order: enter Order ID, search for the matching node, remove it if found, and release its memory.

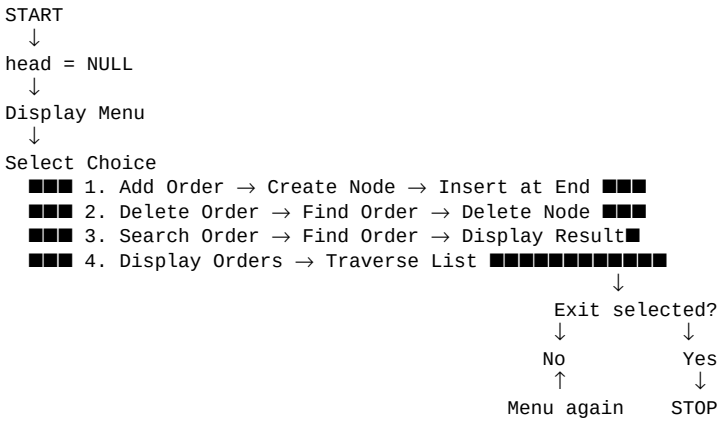
Step 7: For Search Order: enter Order ID and traverse the list until the matching node is found.

Step 8: For Display Orders: start from head and traverse every node, displaying its details.

Step 9: Repeat the menu until Exit is selected.

Step 10: Stop.

6. Flowchart



7. C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
    int orderID;
    string foodItem;
    int quantity;
    Node* next;
};

Node* head = NULL;

// Add a new order
void addOrder()
{
    Node* newNode = new Node;

    cout << "Enter Order ID: ";
    cin >> newNode->orderID;

    cout << "Enter Food Item: ";
    cin >> newNode->foodItem;

    cout << "Enter Quantity: ";
    cin >> newNode->quantity;

    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        Node* temp = head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }

    cout << "Order added successfully!\n";
}

// Delete an order
void deleteOrder()
{
    int id;
    cout << "Enter Order ID to delete: ";
    cin >> id;
    Node* temp = head;
```

```
    Node* prev = NULL;

    while (temp != NULL && temp->orderID != id)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        cout << "Order not found!\n";
        return;
    }

    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;

    delete temp;

    cout << "Order deleted successfully!\n";
}

// Search for an order
void searchOrder()
{
    int id;
    cout << "Enter Order ID to search: ";
    cin >> id;

    Node* temp = head;

    while (temp != NULL)
    {
        if (temp->orderID == id)
        {
            cout << "\nOrder Found!\n";
            cout << "Order ID: " << temp->orderID << endl;
            cout << "Food Item: " << temp->foodItem << endl;
            cout << "Quantity: " << temp->quantity << endl;
            return;
        }

        temp = temp->next;
    }

    cout << "Order not found!\n";
}

// Display all orders
void displayOrders()
{
    if (head == NULL)
    {
        cout << "No pending orders.\n";
        return;
    }

    Node* temp = head;

    cout << "\n----- Pending Orders -----\n";

    while (temp != NULL)
    {
        cout << "Order ID: " << temp->orderID << endl;
        cout << "Food Item: " << temp->foodItem << endl;
        cout << "Quantity: " << temp->quantity << endl;
        cout << "--------------------------\n";

        temp = temp->next;
    }
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== Food Order Management System =====\n";
```

```
        cout << "1. Add Order\n";
        cout << "2. Delete Order\n";
        cout << "3. Search Order\n";
        cout << "4. Display Orders\n";
        cout << "5. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1: addOrder(); break;
            case 2: deleteOrder(); break;
            case 3: searchOrder(); break;
            case 4: displayOrders(); break;
            case 5: cout << "Exiting program...\n"; break;
            default: cout << "Invalid choice!\n";
        }

    } while (choice != 5);

    return 0;
}
```

8. Function Explanation

Function	Purpose
addOrder()	Creates a new node dynamically and inserts the food order at the end of the linked list.
deleteOrder()	Searches for an Order ID, removes the matching node, reconnects the list, and frees memory using delete.
searchOrder()	Traverses the list and displays details when the requested Order ID is found.
displayOrders()	Traverses from head to NULL and displays every pending order.
main()	Displays the menu, accepts the user's choice, and calls the appropriate function repeatedly until Exit.

9. Sample Input / Output

```
===== Food Order Management System =====
1. Add Order
2. Delete Order
3. Search Order
4. Display Orders
5. Exit

Enter your choice: 1
Enter Order ID: 101
Enter Food Item: Pizza
Enter Quantity: 2
Order added successfully!

Enter your choice: 1
Enter Order ID: 102
Enter Food Item: Burger
Enter Quantity: 1
Order added successfully!

Enter your choice: 4

----- Pending Orders -----
Order ID: 101
Food Item: Pizza
Quantity: 2
--------------------------
Order ID: 102
Food Item: Burger
Quantity: 1
--------------------------
```

10. Concepts Used

Singly Linked List • Structure • Pointers • Dynamic Memory Allocation • Insertion • Deletion • Searching • Traversal

11. Advantages

• Dynamic size; nodes can be added as required.

• Efficient insertion and deletion without shifting other elements.

• Demonstrates practical use of pointers and dynamic memory.

• Simple representation of a sequence of pending food orders.

12. Limitations

• Searching is linear and may require traversal of the complete list.

• The current program accepts a single-word food item because it uses cin for input.

• Data is stored only during program execution and is not saved permanently.

13. Applications

The same linked-list concept can be used in restaurant order queues, task management systems, ticket management, print queues, playlists, and other dynamic record-management applications.

14. Conclusion

The Food Order Management System successfully demonstrates the practical implementation of a Singly Linked List in C++. It performs insertion, deletion, searching, and traversal using dynamically allocated nodes. Therefore, it is a suitable Unit III DSA project for understanding linked-list concepts and their real-world application.

15. Future Scope

The project can be extended by adding update-order functionality, customer details, order status, billing, multiple food items per order, sorting, persistent file storage, and a graphical user interface.

16. Viva Questions

Q: What is a singly linked list?
A: A linear data structure in which each node stores data and a pointer to the next node.

Q: Why is dynamic memory allocation used?
A: It allows nodes to be created at runtime according to the number of orders.

Q: What does head store?
A: The address of the first node in the linked list.

Q: What is traversal?
A: Visiting each node sequentially from head until NULL.

Q: What is the time complexity of searching?
A: O(n) in the worst case.

Q: Why is delete used?
A: To release dynamically allocated memory after removing a node.

17. Final Summary

This project uses a Singly Linked List to manage food orders dynamically. Every major operation in the program directly demonstrates Unit III Linked List concepts.